

PROIECT DE PRACTICA

ITCare Helpdesk

Aplicatie desktop pentru gestiunea unui call-center IT. Tichete, asset-uri, rapoarte, SLA — toate intr-un dashboard

care echipa il deschide cu placere.

STUDENT	Covali Alexandr
PROGRAM	Practica ITCare 2026
TEHNOLOGII	.NET 8 · Avalonia 11 · SQL Server · C# v1.0 ·
VERSIUNE	Mai 2026
DOCUMENT	Prezentare tehnica pentru sustinere

v1.0 · PROIECT PRACTICA · MAI 2026

ITCARE HELPDESK

CUPRINS

CAPITOLUL 1

Ce este ITCare Helpdesk?

CAPITOLUL 2

Tehnologiile folosite — explicate simplu

CAPITOLUL 3

Arhitectura aplicatiei pe niveluri

CAPITOLUL 4

Baza de date — tabele, vederi, proceduri

CAPITOLUL 5

Structura codului — fiecare fisier explicat

CAPITOLUL 6

Fluxul utilizatorului (user journey)

CAPITOLUL 7

Design vizual si UX

CAPITOLUL 8

Functionalitati avansate

CAPITOLUL 9

Ghid pentru demo in fata profesorului

CAPITOLUL 10

Intrebari probabile + raspunsuri pregatite

ANEXE

Glosar si comenzi utile

CAPITOLUL 1

Ce este ITCare Helpdesk?

Pe scurt

ITCare Helpdesk este o aplicatie desktop care ajuta o companie IT sa gestioneze centralizat toate problemele tehnice raportate de clientii ei. In termeni simpli: cand cineva suna sau scrie ca "laptopul nu mai porneste", "VPN-ul nu functioneaza" sau "vrem un cont nou pe Office 365", aceasta cerere intra in sistem ca un *tichet*, primeste o prioritate, este atribuita unui tehnician, si este urmarita pana la rezolvare.

De ce este nevoie de asa ceva?

Comaniile IT mici lucreaza adesea cu fisiere Excel sau pe email — "am uitat sa raspund la cererea de luni" se intampla cu tot. ITCare elimina aceasta dezordine: orice cerere devine un rand intr-o lista, are un numar unic (ex. TKT-2026-00042), un termen calculat automat (SLA — Service Level Agreement), si o pista de audit care arata cine ce a facut si cand.

Cine il foloseste

Aplicatia are **trei roluri de utilizatori**, fiecare cu nivel diferit de acces:

Administrator	Acces total. Adauga/modifica utilizatori, configureaza contracte SLA, vede toate datele.
Manager	Vede dashboard-ul complet cu statistici, exporta rapoarte, supravegheaza echipa, dar nu modifica conturi.
Tehnician	Vede si rezolva tichete. Are propriul portofoliu de clienti si vede statisticile lui personale.

Ce face concret aplicatia

Aplicatia este organizata pe **sapte zone principale** accesibile dintr-un meniu lateral. Iata pe scurt fiecare zona:

Dashboard

Pagina principala cu indicatori cheie (KPI): cate tichete sunt deschise acum, cate au depasit SLA-ul, cate au fost rezolvate azi, satisfactia medie a clientilor. Are si doua grafice interactive: un inel (donut) cu distributia tichetelor pe status si un grafic de tip linie cu trendul rezolvarilor in ultimele 7 zile.

Tichete

Lista cu toate tichetele active. Poti filtra dupa prioritate, status, sau cautare libera. Aici creezi tichete noi (buton "+ Tichet nou") sau le inchizi pe cele rezolvate.

Cienti

Galerie cu toti clientii activi — fiecare client are propriul card cu numar de tichete, oras, data contractului.

Tehnicienii

Cartile vizuale cu tehnicienii din echipa — cati tichete a rezolvat fiecare, timpul mediu de rezolvare, rating-ul mediu primit.

Asset-uri

Inventarul de echipamente IT al clientilor (laptopuri, servere, switch-uri, imprimante). Util pentru a sti rapid "ce echipament are clientul X" cand suna.

Rapoarte

Studio de export — generezi un raport PDF, Word sau Excel cu un click. PDF-ul e formatat profesional cu logo, KPI-uri si tabel.

Istoric

Timeline cronologic cu toate activitatile recente — schimbari de status, comentarii, asignari. Poti filtra pe ultimele 7, 14 sau 30 de zile.

De retinut pentru prezentare

ITCare nu este o copie de Excel sau o agenda — este o aplicatie reala care implementeaza **SLA tracking automat** (timpuri maximi de raspuns si rezolvare per tip de contract si prioritate), **audit trail** (orice modificare este inregistrata cu trigger SQL), **autentificare cu OTP** (cod de verificare pe email/sms la sign-up) si **lockout anti-bruteforce**. Adica are toate elementele unui produs business.

CAPITOLUL 2

Tehnologiile folosite

Pentru ca prezentarea sa fie autentica, e bine sa stii ce face fiecare "caramida" pe care este construita aplicatia. Mai jos, fiecare tehnologie cu rolul ei.

.NET 8 — Microsoft

Este "motorul" pe care ruleaza aplicatia — un platforma de dezvoltare creata de Microsoft, stabila, gratuita, cross-platform (merge pe Windows, Linux, macOS). Numarul 8 este versiunea Long Term Support — actualizata pana in 2026. Echivalentul ar fi: cum Microsoft Word are nevoie de Windows ca sa ruleze, aplicatia noastra are nevoie de .NET.

C# — Limbaj de programare

Limbajul propriu-zis in care este scris codul aplicatiei. Sintaxa este similara cu Java sau C++, dar mai moderna si mai sigura. C# este folosit la Stack Overflow, Microsoft Office, GitHub, Unity (jocuri). Are tipare puternice (compilator-ul prinde multe erori inainte de runtime).

Avalonia UI 11 — Interfata utilizator

Framework pentru construirea de aplicatii desktop cu interfata grafica. E o alternativa moderna la WPF (vechi, doar Windows). Avalonia merge cross-platform — aceeasi aplicatie poate fi rulata si pe Linux sau macOS fara modificari. Foloseste un limbaj de descriere a UI numit **XAML** (fisierul cu extensia `.axaml`), care e similar cu HTML.

MVVM — Pattern arhitectural

Acronim pentru **Model-View-ViewModel** — o tehnica de organizare a codului care separa **ce vede utilizatorul** (View) de **logica** (ViewModel) si de **datele brute** (Model). Avantajul: codul devine usor de citit, de testat, si poate fi modificat fara sa strici lucrurile in alta parte. Microsoft promoveaza acest pattern de peste 15 ani.

SQL Server — Baza de date

Sistemul Microsoft de gestiune a bazelor de date — locul unde sunt salvate persistent toate informatiile (utilizatori, tichete, clienti, asset-uri). E o baza de date **relationala**, adica datele sunt organizate in tabele cu randuri si coloane, iar tabele se leaga intre ele prin *chei externe* (foreign keys). SQL Server e standard in industria business.

ADO.NET + Microsoft.Data.SqlClient — Acces la baza de date

Biblioteca prin care codul C# vorbește cu SQL Server. În loc să folosim un ORM "magic" (Entity Framework, care ascunde SQL-ul), am ales să scriem SQL-ul explicit prin proceduri stocate. **De ce:** este o cerință pedagogică (să demonstrezi că știi ADO.NET), este mai rapid, și la review se vede clar ce comenzi SQL se execută.

CommunityToolkit.Mvvm — Toolkit MVVM

Pachet Microsoft care reduce boilerplate-ul din ViewModels prin source generators. Exemplu: în loc să scrii 10 linii de cod ca să notifici UI-ul când o variabilă se schimbă, scrii o singură linie:

```
[ObservableProperty] private string _name; și toolkit-ul generează restul automat.
```

SkiaSharp — Motor de desen 2D

Folosit pentru a desena chart-urile custom (donut, sparkline) și particulele animate de pe splash/login. SkiaSharp este wrapper-ul .NET peste motorul grafic **Skia** (cel din Google Chrome și Android). Foarte rapid pe GPU.

QuestPDF — Generator de PDF

Biblioteca modernă pentru a crea rapoarte PDF dintr-un API fluent (înțelegibil, ca un text). Folosită la pagina Rapoarte pentru a genera raportul cu KPI-uri și tabel.

ClosedXML — Generator Excel

Biblioteca pentru a crea fișiere .xlsx programatic. API mai prietenos decât alternativele (EPPlus, OpenXML pur).

DocumentFormat.OpenXML — Generator Word

Biblioteca oficială Microsoft pentru a crea fișiere .docx. Permite control fin pe styling.

Microsoft.Extensions.DependencyInjection — Container DI

Sistemul de "dependency injection" — modul prin care aplicația construiește și leagă toate serviciile (autentificare, navigare, repo-uri) între ele în mod centralizat. Evită ca fiecare clasă să-și construiască dependentele singură.

CAPITOLUL 3

Arhitectura aplicației

Aplicația este organizată pe **patru niveluri (layers)**, ca o tarta cu felii suprapuse. Fiecare nivel are o singură responsabilitate. Când utilizatorul face un click, comanda calătorește de sus în jos prin niveluri până la baza de date, apoi răspunsul urcă înapoi.

UI Layer (Views) – Fisierele .axaml + .axaml.cs

Acesta este "obrazul" aplicatiei — tot ce vede si atinge utilizatorul. Butoane, campuri text, dropdown-uri, ferestre.

Definite in XAML (un dialect de XML descriptiv).

**Presentation Layer (ViewModels)** – Fisierele *ViewModel.cs

Logica de prezentare. Stie ce trebuie afisat, ce face fiecare buton, cand sa afiseze o eroare. Nu "stie" cum arata UI-ul (View-ul) — comunica prin *data binding*.

**Service Layer** – Fisierele din Services/

Aici locuieste logica de business si infrastructura: autentificare, sesiune, OTP, navigare, generare rapoarte. Sunt clase "singleton" reutilizate in toata aplicatia.

**Repository Layer** – Fisierele *Repository.cs

Punctul unic de acces la baza de date. Fiecare entitate (tichet, client, asset) are propriul repository care contine apelurile SQL.

**Database Layer** – Fisierele .sql din database/

Stratul de stocare. Microsoft SQL Server cu tabele, view-uri, proceduri stocate.



Un exemplu concret de flow

Sa zicem ca utilizatorul apasa "+ Tichet nou" pe pagina Tichete si completeaza formularul. Iata ce se intampla, pas cu pas:

1**View**

Butonul + Tichet nou are atasata o comanda. Cand utilizatorul da click, View-ul cere ViewModel-ului sa execute aceasta comanda.

2**ViewModel**

TicketsViewModel deschide o fereastră modală (CreateTicketWindow) si asteapta rezultatul.

- 3 **ViewModel modal**
CreateTicketViewModel incarca listele de clienti, categorii si tehnicieni in paralel din baza de date (prin repository-uri).
- 4 **User**
Utilizatorul completeaza titlul, alege clientul, categoria, prioritatea, apoi apasa Creeaza tichet.
- 5 **Repository**
TicketRepository.OpenTicketAsync() este apelat. El construiește un SqlCommand cu procedura stocata sp_DeschideTichet si parametrii completati.
- 6 **Database**
Procedura SQL genereaza un numar unic (ex. TKT-2026-00018), insereaza randul in tabela Tichete, ataseaza un SLA automat dupa prioritate, si scrie o intrare in tabela IstoricActivitate.
- 7 **Repository → ViewModel**
Numarul de identitate al noului tichet este returnat. CreateTicketViewModel emite un eveniment TicketCreated.
- 8 **View modal**
Fereastra modala se inchide cu rezultatul.
- 9 **ViewModel parinte**
TicketsViewModel reincarca lista de tichete din DB ca sa includa noul tichet.
- 10 **UI**
Lista se actualizeaza automat datorita data binding-ului. Apare si un toast verde in coltul de jos: "Tichet creat: TKT-2026-00018".

Avantajul arhitecturii pe niveluri

Pentru ca fiecare nivel este izolat, daca trebuie sa schimbam baza de date (ex. mutam pe PostgreSQL), atingem doar Repository Layer. UI-ul nu se schimba. Daca vrem sa adaugam o aplicatie mobila, putem reutiliza ViewModels si Services — schimbam doar View-urile. Asta inseamna cod sustenabil.

CAPITOLUL 4

Baza de date

Baza de date se numeste **ITCareHelpdesk** si ruleaza pe Microsoft SQL Server. Este construita prin trei scripturi SQL care se ruleaza in ordine:

01_main.sql — script SQL

Schema principala: 9 tabele, 2 view-uri, 9 proceduri stocate. Sterge baza existenta (daca exista) si recreaza totul de la zero — scriptul e idempotent, il poti rula de cate ori vrei.

02_extensions.sql — script SQL

Extensii de securitate: 3 tabele suplimentare (OtpCodes, LoginAttempts, AuditTrail), 4 proceduri (sp_RequestOtp, sp_VerifyOtp, sp_SignUp, sp_LogLoginAttempt), o functie scalara (fn_IsAccountLocked), un trigger (trg_Tichete_Audit) si un view (vw_DashboardKpi).

03_seed.sql — script SQL

Date demo: 12 clienti, 8 tehnicieni, 11 conturi de utilizatori, 15 asset-uri, 17 tichete cu varietate de statusuri/prioritati. Hash-urile parolelor sunt generate dinamic prin HASHBYTES.

setup-db.ps1 — script PowerShell

Script PowerShell care le ruleaza pe toate trei in ordine automata. Tot ce trebuie sa stii sa rulezi.

Tabelele principale

Mai jos, fiecare tabela cu rolul ei si coloanele cele mai importante.

Utilizatori — Conturi de aplicatie

Stocheaza userii care se pot autentifica. Coloane cheie: **username** (unic), **parola_hash** (SHA256 in hex), **rol** (Admin/Manager/Technician), **tehnician_id** (legatura optionala catre tabela Tehnicieni — un user poate fi sau nu tehnician). Constrangerile UNIQUE pe username si email previn duplicate.

Tehnicieni — Membrii echipei IT

Resursele umane operationale. Coloane: **cod_tehnician** (ex. ITC-T001), nume, prenume, email, telefon, specializare (Network, Hardware etc.), nivel (Junior, Mid, Senior, Lead), data angajare.

Clienti — Companii deservite

Lista companiilor sub contract. Coloane: nume_companie (unic), CUI, industrie, oras, telefon, email_contact, data_contract, activ (bool).

Departamente + Categori

 — Taxonomia tichetelor

Doua tabele legate prin FK. Departamente are 4 zone: Network, Hardware, Software, Security. Categoriile sunt sub-temele fiecarui departament (ex. VPN sub Network, Imprimante sub Hardware).

ContracteSLA

 — Timpi maximi de raspuns/rezolvare

Matrice 4×4: 4 niveluri de contract (BRONZE/SILVER/GOLD/PLATINUM) × 4 prioritati (CRITICAL/HIGH/MEDIUM/LOW). Fiecare combinatie are timp_raspuns_ore si timp_rezolvare_ore — referintele dupa care un tichet e considerat "depasit SLA".

Assets

 — Echipamente IT

Inventar de echipamente apartinatoare clientilor: laptopuri, servere, switch-uri etc. Coloane: cod_asset (unic), denumire, tip, producator, model, serial_number, client_id (FK), locatie, status, garantie_pana.

Tichete

 — Inima sistemului

Tabela cea mai bogata. Coloane: numar_tichet (TKT-2026-00001), titlu, descriere, FK catre client + categorie + tehnician + sla, prioritate, status, tip, data_deschidere (auto), data_rezolvare, data_inchidere, ore_lucrate, rating_client (1-5), creat_de + inchis_de (FK Utilizatori). Are 4 indexuri pentru query-uri rapide.

IstoricActivitate

 — Log de business

Inregistreaza fiecare actiune semnificativa pe un tichet: schimbare status, comentariu, asignare, time-tracking. Folosit pentru pagina Istoric.

OtpCodes

 — Coduri OTP (sign-up)

Codurile de 6 cifre generate la inregistrarea unui cont nou. Expira in 5 minute, se sterg dupa folosire. Limita de 5 incercari per cod.

LoginAttempts

 — Tracking incercari de autentificare

Logheaz■ fiecare incercare de login (reusita sau nu) cu username, IP, timestamp. Functia fn_IsAccountLocked se bazeaza pe aceasta tabela pentru a bloca temporar conturile cu prea multe esecuri.

AuditTrail

 — Audit tehnic generic

Stocheaza modificari capturate de trigger-ul trg_Tichete_Audit. Cand statusul unui tichet se schimba, trigger-ul automat scrie aici un rand cu old value, new value, user-ul care a facut modificarea, timestamp.

View-uri (vederi)

View-urile sunt "interogari pre-impachetate" pe care le poti folosi ca pe tabele. In ITCare avem trei view-uri:

vw_TicheteActive — view SQL

Returneaza toate tichetele care NU sunt inchise (status NOT IN CLOSED, CANCELLED). Combina datele din Tichete + Clienti + Categori + Departamente + Tehnicieni + ContracteSLA intr-un singur rand pe tichet, cu coloane denormalizate (numele clientului, numele tehnicianului etc.). Asta inseamna ca aplicatia C# face un singur SELECT in loc de 5 JOIN-uri.

vw_TicheteIntarziate — view SQL

Subset al vw_TicheteActive — doar tichetele care depasesc timpul SLA de rezolvare. Folosit pentru KPI-ul "Tichete intarziate" si pentru raportul de SLA breach.

vw_DashboardKpi — view SQL

Agregeaza intr-un singur rand cele 4 metrice principale (tichete deschise, intarziate, rezolvate azi, clienti activi). E un alias pentru o serie de subqueries — convenient cand vrei toate KPI-urile dintr-un singur apel.

Procedurile stocate (Stored Procedures)

Sunt "functii" scrise in SQL care traiesc in baza de date. Aplicatia le apeleaza in loc sa trimita query-uri brute. Avantajele: **(1)** performanta (sunt compilate o singura data), **(2)** securitate (parametrii sunt scapati automat, fara SQL injection), **(3)** mentenabilitate (daca schimbi logica, schimbi un singur loc).

sp_Login — stored procedure

Autentificare. Primeste username + hash parola. Returneaza succes/elec, user_id, rol si mesaj. Logheaza incercarile in LoginAttempts si blocheaza dupa 5 esecuri in 15 minute.

sp_SignUp — stored procedure

Creeaza un cont nou (dupa verificare OTP). Validari: username minim 3 caractere, parola_hash minim 32 caractere (SHA256), email format valid.

sp_RequestOtp — stored procedure

Genereaza si stocheaza un cod OTP pentru un identifer (email/telefon). Codurile vechi sunt invalidate automat.

sp_VerifyOtp – stored procedure

Verifica un cod OTP. Expiry 5 minute, max 5 incercari per cod.

sp_DeschideTichet – stored procedure

Insereaza un tichet nou. Genereaza automat numarul (TKT-YYYY-NNNNN), ataseaza SLA-ul potrivit pentru prioritate, inregistreaza in IstoricActivitate.

sp_InchideTichet – stored procedure

Marcheaza un tichet ca rezolvat/inchis. Updateaza data_inchidere, ore_lucrate, rating_client, scrie in IstoricActivitate.

sp_GetTicheteCritice – stored procedure

Returneaza tichete cu prioritate CRITICAL/HIGH care nu sunt inca rezolvate, sortate dupa vechime.

sp_GetTicheteIntarziate – stored procedure

Returneaza tichetele cu SLA depasit. Accepta filtre optionale pe departament si prioritate.

sp_GetIstoricActivitate – stored procedure

Returneaza istoricul de activitate pe un numar de zile inapoi. Filtre optionale: client_id si tichet_id.

sp_GetStatisticiTehnicieni – stored procedure

Calculeaza pentru fiecare tehnician: tichete totale, rezolvate, deschise, timp mediu de rezolvare, rating mediu.

sp_GetTopTehnicieni – stored procedure

Acelasi shape ca sp_GetStatisticiTehnicieni dar cu un loc_clasament (ROW_NUMBER) si limita TOP N.

sp_GetTimpMediiPerCategorie – stored procedure

Statistici de performanta per categorie de tichet — total, procent rezolvate, timp mediu, total ore lucrate.

CAPITOLUL 5

Structura codului — fisier cu fisier

Aceasta este harta completa a codului. Pentru fiecare fisier ai numele, rolul lui si o explicatie cu cuvinte simple. Daca te intreaba profesorul "ce face acest fisier?", rasfoiesti aici si gasesti raspunsul.

Fisiere de configurare la nivel de proiect

ITCareHelpdesk.sln — solution file

Un fisier text care spune Visual Studio / Rider "acesta este un proiect, iata partile lui". Doar metadata, nu cod executabil. Echivalentul unui "index" de proiect.

ITCareHelpdesk.App.csproj — project file MSBuild

Reteta de compilare. Contine: pe ce versiune .NET ruleaza (net8.0), ce pachete NuGet se folosesc (Avalonia, QuestPDF, SqlClient etc.), ce este OutputType (WinExe — adica produs final este un .exe Windows).

app.manifest — Windows manifest

Fisier XML pentru Windows care declara: ce versiuni de Windows e compatibila aplicatia, daca cere drepturi de administrator (NU cere), cum sa scaleze pe ecrane high-DPI.

appsettings.json — configurare runtime

Setari care pot fi schimbate fara recompilare: connection string-ul SQL Server, modul OTP (simulation = afisat in toast, vs real = trimis prin SMS), timpul de expirare OTP, numarul maxim de incercari de login.

Program.cs — punctul de intrare al aplicatiei

Locul de unde Windows lanseaza aplicatia. Cele 30 de linii configureaza Avalonia (activeaza ReactiveUI, foloseste fontul Inter, prinde crash-urile globale si le scrie intr-un fisier log local).

App.axaml — configurare globala Avalonia

Defineste tema (Dark), include stilurile globale (Palette, DarkTheme, Controls) si incarca paleta de resurse. Echivalentul unui "global stylesheet".

App.axaml.cs — logica de pornire

Configureaza Dependency Injection (DI) — adica registreaza toate serviciile si ViewModels in container, ca acestea sa fie disponibile in toata aplicatia. Tot aici se decide ca prima fereastra deschisa este SplashScreen.

Folder: Assets/

`logo-itcare.svg` — logo vector

Logoul aplicației în format vectorial (SVG) — scalează fără pierdere de calitate pe orice DPI. Afisat pe splash, login și în colțul stanga-sus al ferestrei principale.

Folder: Themes/

Cele trei fișiere XAML care definesc **aspectul** întregii aplicații. Schimbă o culoare aici și se schimbă peste tot.

`Palette.axaml` — paleta + tipografie

Definițiile de bază: culorile fundalului (#06070D — negru-albastru "deep space"), culoarea accent (#43BAFF — cyan ITCare), nuanțele de text (alb, gri, mut). Defineste și fonturile (Inter pentru text, Cascadia Mono pentru cod), scarile de fonturi (10, 12, 14, 20, 28, 42, 64 pixel), și *corner radius*-urile.

`DarkTheme.axaml` — stiluri pentru text & ferestre

Stilurile globale aplicate automat pe Window, TextBlock și clasele de tipografie (.h1, .h2, .h3, .body, .label, .display, .mono). Aici am pus și **animatia de gradient în titluri** — text-ul mare "curge" prin cyan → indigo → purple → cyan în 8 secunde.

`Controls.axaml` — stiluri pentru controale

Stilurile pentru Button (cu variante .primary, .ghost, .danger), TextBox (cu linie subțilă sub care se aprinde la focus), ListBox, DataGrid, ComboBox, și clasele de containere (.card, .card-glow, .pill, .flat-rows). Defineste și *transitions*-urile (animatii smooth pe hover).

Folder: Controls/

Controale custom desenate manual prin SkiaSharp — chart-urile și efectele vizuale.

`ParticleField.cs` — particule animate

Desenează ~60 puncte cyan care plutesc în sus și dispar, pe canvas Skia. Apare pe splash screen și pe panelul stâng al ferestrei de login. Creează atmosfera "mission control" — puncte mici care se mișcă în fundal. Implementare directă pe GPU canvas (60 FPS, fără impact de performanță).

Converters.cs — convertoare pentru data binding

11 "functii" mici care transforma valori intre tipuri — folosite in XAML. Exemple: **ProgressToWidth** (transforma un procent 0..1 in pixeli), **PriorityToBrush** (prioritate text → culoare rosie/portocalie), **StatusToBrush**, **EnumEquals** (compara un enum cu un string), **FirstInitial** (extrage prima litera dintr-un nume pentru avatar), **RatingToStars** (numar → ★★★★★), **DateToRelative** ("acum 5 min").

SparklineChart.cs — chart linie pentru trend

Control custom care primeste o lista de valori (numar de tichete rezolvate per zi) si deseneaza un grafic de tip linie cu aria de sub linie umbrita cyan, puncte la fiecare zi si etichete de data. Folosit pe Dashboard la KPI-ul "Trend rezolvari 7 zile".

DonutChart.cs — chart inel pentru distributie

Control custom care primeste o lista de status-uri + numar de tichete si deseneaza un inel segmentat cu culoarea fiecarui status. In mijloc afiseaza totalul. Folosit pe Dashboard la "Distributie status".

Folder: Models/

DomainModels.cs — structuri de date

Un singur fisier care contine toate **POCO-urile** (Plain Old C# Objects) — clase simple care reprezinta entitatile aplicatiei: *AppUser*, *Ticket*, *Client*, *Technician*, *Asset*, *CategoryStat*, *HistoryEntry*, *DashboardKpi*, *CategoryOption*, *TechnicianOption*, *StatusBucket*, *DailyResolved*. Sunt definite ca **records** (imutabile, generate automat cu equals/hashcode/tostring). Aceste obiecte sunt "vehiculul" prin care datele circula intre Repository → ViewModel → View.

Folder: Services/

Serviciile sunt "angajatii" aplicatiei — clase care fac munca de business sau infrastructura. Sunt construite o singura data (singleton) si reutilizate peste tot.

DatabaseService.cs — wrapper SQL

Punctul unic prin care orice repository ajunge la baza de date. Expune metode utile:

OpenConnectionAsync (deschide o conexiune), *QueryAsync<T>* (executa un query si materializeaza rezultatele intr-o lista de obiecte prin functii mapper), *ScalarAsync*, *ExecuteAsync*, *TestConnectionAsync* (folosit pe splash). Citeste connection string-ul din appsettings.json.

AuthService.cs – autentificare si signup

Apeleaza `sp_Login` si `sp_SignUp`. Contine functia `HashPassword` care calculeaza SHA256 al unei parole si returneaza hash-ul ca string hex lowercase — exact formatul stocat in baza de date. Returneaza tuple (rezultat, user, mesaj).

SessionService.cs – sesiunea utilizatorului curent

O clasa simpla care tine in memorie userul autentificat. Are `IsAuthenticated`, `IsAdmin`, `IsManager`. Cand utilizatorul iese, se cheama `SignOut()` care goleste sesiunea.

OtpService.cs – generare si verificare OTP

Genereaza coduri OTP de 6 cifre folosind `RandomNumberGenerator` (cryptographically secure, nu `Math.Random` care e predictibil). Le stocheaza in DB prin `sp_RequestOtp`. Verifica prin `sp_VerifyOtp`. In modul "simulation" (configurat din `appsettings.json`), arata codul direct intr-un toast — pentru demo. In productie reala s-ar trimite prin Twilio/SMTP.

ToastService.cs – notificari toast

Cooda observabila de notificari afisate in coltul dreapta-jos. Are metode `ShowInfo`, `ShowSuccess`, `ShowWarning`, `ShowError`, `ShowOtp`. Fiecare toast dispare automat dupa 5 secunde (30s pentru OTP).

NavigationService.cs – schimbarea paginilor

Centralizeaza navigarea intre paginile din `MainWindow`. Are un enum `AppPage` (Dashboard, Tickets, Clients, Technicians, Assets, Reports, History) si o metoda `NavigateTo(page)` care cere `ViewModel`-ul potrivit din DI si-l plaseaza in `CurrentView`. `View`-ul reactioneaza automat prin binding.

ReportService.cs – generare rapoarte

Trei metode de export: `ExportTicketsToWorldAsync` (OpenXML), `ExportTicketsToExcelAsync` (ClosedXML), `ExportTicketsToPdfAsync` (QuestPDF). Fiecare ia o lista de tichete si o cale de output, scrie fisierul si returneaza calea. PDF-ul are layout boutique cu header negru, KPI strip, tabel cu pill-uri colorate si paginare.

TicketRepository.cs – acces la tabela Tichete

Metode: `GetActiveAsync` (citeste `vw_TicheteActive`), `GetOverdueAsync` (`sp_GetTicheteIntarziate`), `GetCriticalAsync` (`sp_GetTicheteCritice`), `OpenTicketAsync` (`sp_DeschideTichet`), `CloseTicketAsync` (`sp_InchideTichet`).

ClientRepository.cs — acces la tabela Clienti

O singura metoda principala — `GetAllAsync` — care returneaza toti clientii activi cu numarul de tichete per client (subquery).

AssetRepository.cs — acces la tabela Assets

`GetAllAsync` cu JOIN pe Clienti pentru a returna direct numele clientului, nu doar ID-ul.

HistoryRepository.cs — acces la IstoricActivitate

`GetAsync` cu parametri opționali (client, tichet, numar zile inapoi).

StatsRepository.cs — statistici pentru Dashboard

Cele mai "agregate" metode: `GetKpiAsync` (un singur query cu mai multe subqueries pentru cele 6 KPI-uri), `GetTechniciansAsync`, `GetTopTechniciansAsync`, `GetCategoryStatsAsync`, `GetStatusDistributionAsync` (pentru donut chart), `GetResolvedByDayAsync` (pentru sparkline).

CategoryRepository.cs — acces categorii

Folosit doar in dialogul de creare tichet — pentru a popula dropdown-ul cu categoriile disponibile (incluzand departamentul ca prefix vizual).

Folder: ViewModels/

Cate un ViewModel pentru fiecare pagina sau fereastră — el contine starea (proprietati) si comportamentul (comenzi) pe care le foloseste UI-ul.

ViewModelBase.cs — clasa de baza

O clasa abstracta de la care toate celelalte ViewModels mostenesc. Are doua proprietati comune: `IsBusy` (true cand se incarca date) si `BusyMessage`. Mosteneste de la `ObservableObject` din `CommunityToolkit` care implementeaza automat `INotifyPropertyChanged` (mecanismul prin care UI-ul "observa" modificarile).

SplashViewModel.cs — logica splash screen

Ruleaza secventa de pornire: verifica conexiunea la SQL Server, inainte de a permite trecerea la login. Daca DB-ul nu raspunde, afiseaza eroarea reala (connection string + mesaj de la `SqlClient`) — debug rapid. Foloseste o animatie ease-out pentru progress bar ca sa para natural, nu robotic.

LoginViewModel.cs — login + signup + OTP

Un mic state machine cu trei panouri: *SignIn* (autentificare), *SignUp* (inregistrare cont nou cu validari client-side), *OtpVerify* (introducere cod OTP). Tranzitiile sunt cross-fade animate. Cand login-ul reuseste, emite evenimentul `SignedIn` ascultat de `Window`.

MainWindowViewModel.cs — shell-ul aplicatiei

Orchestreaza sidebar-ul si content host-ul. Are o lista de `NavItem` (titlu + glyph + pagina destinatie). Cand utilizatorul da click pe un nav-item, `NavigationService.NavigateTo` schimba pagina curenta. Expune si `SessionService` (pentru user-pill) si `ToastService` (pentru queue de toast-uri).

DashboardViewModel.cs — logica dashboard

Ruleaza 6 incarcari paralele (`Task.WhenAll`): KPI, top tehnicieni, tichete critice, statistici categorii, distributia status, trend rezolvari. Fiecare incarcare are propriul try/catch, deci o cadere intr-un singur SP nu blanchete toata pagina.

TicketsViewModel.cs — logica pagina Tichete

Master list cu filtru live (prioritate, status, cautare libera, "doar ale mele"). Tine si lista "master" (`_all`) pentru a putea reseta filtru fara round-trip nou la DB. Comenzi: `Load`, `CloseTicket`. Eveniment de notificare cand se creeaza un tichet nou (reincarca lista).

CreateTicketViewModel.cs — logica dialog creare tichet

Formularul de creare. Incarca paralel listele de clienti, categorii, tehnicieni la pornire. Validare client-side: titlu minim 6 caractere, client si categorie obligatorii. La submit, apeleaza TicketRepository.OpenTicketAsync si emite evenimentul TicketCreated.

ClientsViewModel.cs — logica pagina Clienti

Master list cu filtru de cautare (nume, oras, industrie).

TechniciansViewModel.cs — logica pagina Tehnicieni

Incarca statisticile per tehnician din sp_GetStatisticiTehnicieni.

AssetsViewModel.cs — logica pagina Asset-uri

Master list + filtru pe tip echipament + cautare libera. Tipurile sunt extrase dinamic din date (nu hardcodate) — daca apare un tip nou in DB, apare automat in dropdown.

ReportsViewModel.cs — logica pagina Rapoarte

Trei comenzi: ExportPdf, ExportWord, ExportExcel. Fiecare deschide un dialog de salvare (Avalonia FilePicker), apoi cheama ReportService cu calea aleasa.

HistoryViewModel.cs — logica pagina Istoric

Reincarca lista cu activitati pe intervalul ales (7/14/30 zile).

Folder: Views/

Pentru fiecare ViewModel, un fisier `.axaml` (XAML — descrierea vizuala) si un `.axaml.cs` (code-behind — comportamente specifice ferestrei).

`SplashWindow.axaml (+.cs)` — View Avalonia

splash screen — fereastra borderless 640×420 cu logo central, particule SkiaSharp, progress bar subtire cu glow cyan. Are si un overlay de eroare daca DB-ul nu raspunde.

`LoginWindow.axaml (+.cs)` — View Avalonia

fereastra de autentificare. Doua coloane: stanga = hero cu logo + titlu display animat + particule; dreapta = unul din trei panouri (SignIn, SignUp, OtpVerify) ales prin convertor EnumEquals. Auto-advance pe casutele OTP.

`MainWindow.axaml (+.cs)` — View Avalonia

fereastra principala. Layout: 240px sidebar (logo, nav items, user-pill jos) + content host la dreapta. Top bar cu search palette si window controls custom (minimize/maximize/close).

`DashboardView.axaml (+.cs)` — View Avalonia

Dashboard cu sectiuni: header, KPI strip (6 carduri), row de 3 carduri (Donut + Sparkline + Top tehnicieni), card cu Tichete critice, card cu Distributie pe categorii (bare orizontale).

`TicketsView.axaml (+.cs)` — View Avalonia

Lista de tichete tip tabel cu sticky header. Toolbar cu filtre. Buton primary "+ Tichet nou" sus-dreapta care deschide dialog modal.

`CreateTicketWindow.axaml (+.cs)` — View Avalonia

dialog modal pentru creare tichet — 720×780, centrat pe MainWindow. Formular cu titlu, descriere, client, categorie, prioritate, tip.

`ClientsView.axaml (+.cs)` — View Avalonia

galerie cu carduri 320×160 in WrapPanel — fiecare client are avatar text, numar de tichete, locatie, data contract.

`TechniciansView.axaml (+.cs)` — View Avalonia

galerie cu "trading cards" 340px — fiecare tehnician are cod, nume, specializare, 3 metrici (rezolvate/active/medie ore), rating stele.

AssetsView.axaml(+.cs) – View Avalonia

tabel cu coloane: cod, denumire, tip, producator, client, status (pill colorat), garantie.

ReportsView.axaml(+.cs) – View Avalonia

Studio cu 3 carduri masive (PDF/Word/Excel) — fiecare cu iconita literala mare in cerc colorat, descriere, buton de export.

HistoryView.axaml(+.cs) – View Avalonia

timeline cronologic cu linia verticala si puncte. Pe fiecare punct, un card cu detalii ale activitatii.

CAPITOLUL 6

Fluxul utilizatorului

Iata calatoria tipica a unui utilizator cand deschide aplicatia, in ordine cronologica:

- 1. Lansare aplicatie**
Utilizatorul face dublu-click pe `ITCareHelpdesk.exe` sau porneste din Rider cu F5. Windows incarca .NET runtime si trece controlul lui Program.Main.
- 2. Splash Screen**
Apare o fereastra borderless 640×420 cu logo ITCare central si particule cyan plutind in fundal. In timp ce o vede, aplicatia: (1) verifica conexiunea la SQL Server, (2) face warmup la servicii, (3) incrementeaza progress bar-ul cu ease-out. Total ~2 secunde. Daca DB-ul nu raspunde, apare un overlay cu eroare detaliata si buton de inchidere.
- 3. Login Window**
Splash-ul se inchide si apare LoginWindow (1100×720). Doua coloane: in stanga, logoul + titlu mare animat (cyan → purple → cyan), in dreapta formularul de login. Utilizatorul completeaza username + parola si apasa *Intra in cont*. Daca e cont nou, click pe *Creeaza un cont nou* → completeaza datele → primeste un cod OTP de 6 cifre in toast (modul simulation) → introduce codul (pasted-ul se distribuie automat pe toate 6 casutele) → cont creat → intoarcere la SignIn.
- 4. Autentificare**
Parola e trecuta prin SHA256, apoi se cheama `sp_Login`. Daca match-uieste hash-ul stocat, se logheaza ca succes (`sp_LogLoginAttempt`), userul e pus in `SessionService`, apare un toast verde "Bun venit, X" si fereastra se inchide.
- 5. Main Window**
Se deschide MainWindow (1480×900). Sidebar in stanga (logo + 7 nav items + user pill jos), top bar in dreapta (search palette + window controls custom). Pagina initiala: **Dashboard**.

6.

Dashboard

Sase incarcari paralele aduc: KPI-uri (6 carduri), top 5 tehnicieni, tichete critice, statistici pe categorii, distributie status (donut), trend rezolvari 7 zile (sparkline). Daca o cerere pica, restul tot apar — fail-safe.

7.

Navigare laterala

Click pe un nav item (ex. *Tichete*) → NavigationService cere ViewModel-ul potrivit din DI → MainWindow detecteaza schimbarea prin binding → cross-fade 180ms catre noul View.

8.

Creare tichet

Pe pagina Tichete, click pe **+ Tichet nou** → modal centrat pe MainWindow → completare formular → submit → sp_DeschideTichet → modal se inchide → lista se refresh → toast verde.

9.

Generare raport

Pe pagina Rapoarte, click pe **Genereaza .pdf** → file picker → alegi locatia → QuestPDF construiesc PDF-ul cu header brand, KPI strip, tabel cu pill-uri colorate, footer cu paginare → toast verde "Export gata" → fisierul se deschide cu OS-ul default.

10.

Iesire

Click pe iconul ■ langa avatar in sidebar → SessionService.SignOut → LoginWindow se redeschide → MainWindow se inchide. Pentru a inchide complet aplicatia, butonul ✕ din coltul dreapta-sus.

CAPITOLUL 7

Design vizual si UX

Aplicatia nu arata ca un dashboard SaaS plictisitor pe alb-albastru. Are personalitate. Iata principiile vizuale aplicate.

Paleta de culori

Constructia paletei sta pe trei axe: **backgrounds** in nuante negru-albastru (NU negru pur, pentru ca arata ieftin pe ecrane OLED), **surfaces** in straturi de "sticla" cu opacitate diferita (ierarhie vizuala fara linii grele) si **accent** cyan electric (#43BAFF — exact cyan-ul brandului ITCare).

#06070D	Background	Fundal principal ("void")
#10141F	Surface 1	Carduri si suprafete principale
#161B28	Surface 2	Header-ele si stripa de table
#43BAFF	Accent	Cyan ITCare — culoarea brand
#FF3B5C	Critical	Erori, prioritate CRITICAL
#2DD4BF	Success	Stari finalizate
#FBBF24	Warning	IN_PROGRESS

Tipografia

Fontul principal este **Inter** — un font sans-serif modern, optimizat pentru ecrane, creat special pentru interfete. Are 7 dimensiuni in aplicatie, intr-o scara matematica (perfect fifth, ratia 1.5): 10 → 12 → 14 → 20 → 28 → 42 → 64 pixel. Asta inseamna ca ierarhia vizuala se simte naturala — fiecare nivel e "clar mai mare" decat precedentul. Pentru cod si numere mono spatiate folosim **Cascadia Mono**.

Glassmorphism

Tehnica vizuala in care suprafetele par "sticla translucida" — opacitate mica pe alb peste fundal intunecat, plus border subtle. Folosita pe carduri, pill-uri, side-panel-uri. Da senzatie de profunzime si straturi, fara sa fie ostentativ.

Animatii

Animatiile au scop, nu sunt eye-candy gratuit. Cateva exemple:

Particule pe splash/login	Misca ochiul de la stanga la dreapta — implementeaza un Z-pattern de citire natural.
Cross-fade intre pagini	180ms — suficient cat sa se simta tranzitia, putin cat sa nu enerveze.
Spinner pulsatoriu	Cerc cyan care "respira" pe Opacity. In loc de rotatie (care necesita animator special), folosim pulse — mai simplu, look modern.
Gradient pe titluri	Foreground-ul titlurilor mari curge prin cyan → indigo → purple → cyan in 8 secunde. Memorabil.
Hover pe nav items	Backgroundul listei laterale se schimba subtle pe hover; pe item selectat apare o bara verticala cyan in stanga.
Press feedback pe butoane	Translatie de 1px in jos — feeling natural de "apasat".

CAPITOLUL 8

Functionalitati avansate

Autentificare cu OTP la sign-up

Cand un utilizator nou se inregistreaza, dupa completarea formularului primeste un cod de 6 cifre. In modul simulation acest cod apare intr-un toast (pentru demo); in productie ar fi trimis prin SMS (Twilio) sau email (SMTP). Codul expira in 5 minute si se invalideaza automat dupa folosire. UI-ul are 6 casute separate cu auto-advance — daca lipesti codul intreg in prima casuta, el se distribuie pe toate sase, ca pe iOS.

Anti-bruteforce / logout

Fiecare incercare de login este inregistrata in tabela LoginAttempts. Functia scalara fn_IsAccountLocked verifica daca un user a avut 5 incercari esuate in ultimele 15 minute — daca da, sp_Login refuza autentificarea cu un mesaj clar ("Cont blocat temporar. Asteapta 15 minute."). Protejeaza impotriva atacurilor de tip dictionary.

SLA tracking automat

Fiecare tichet primeste automat un SLA la creare, pe baza prioritatii lui (PLATINUM CRITICAL = raspuns in 1h + rezolvare in 4h, BRONZE LOW = raspuns 48h + rezolvare 168h). View-ul vw_TicheteActive calculeaza la fiecare interogare daca tichetul a depasit SLA (sla_depasit = 1 daca ore_deschis > timp_rezolvare_ore). In UI, asta apare ca un fundal rosu subtil pe coloana "Ore".

Audit trail automat

Cand un tichet isi schimba statusul (INSERT/UPDATE), trigger-ul SQL trg_Tichete_Audit scrie automat un rand in AuditTrail cu vechea valoare si noua. Asta inseamna ca chiar daca cineva modifica direct in DB (cu un query SQL manual), modificarea este capturata. Audit-ul este "transparent" — nu poate fi ignorat din cod.

Export multi-format

Acelasi set de tichete poate fi exportat in PDF (cu QuestPDF — layout boutique, KPI strip, tabel zebra cu pill-uri colorate pentru prioritati), Word (OpenXML — tabel simplu pentru imprimare/sedinte), sau Excel (ClosedXML — pentru analiza si pivot tables). Fiecare format are stilul lui propriu.

Toast queue cu auto-dismiss

Notificari non-modale care apar in coltul dreapta-jos al MainWindow. Stiva, nu se suprapun. Fiecare dispare automat dupa 5 secunde (30 pentru OTP, care e mai important). Animatie de fade subtle. Toast-urile sunt observable din toate ViewModel-urile prin ToastService.

Search palette in top bar

Camp de cautare in top-bar-ul MainWindow cu hint vizual "Ctrl K" — pregatit pentru viitoarea integrare cu command-palette tip Linear (deschide tichet TKT-X, creeaza tichet pentru client Y, etc.). Momentan e doar filtru live pe pagina curenta.

Chart-uri custom (Skia)

Donut chart si sparkline desenate direct pe canvas — fara biblioteca externa de charting. Avantaje: ~80 linii cod per chart, control fix peste stilul aplicatiei, no extra dependencies. Donut-ul calculeaza unghiurile sectoarelor cu arctan, sparkline-ul deseneaza polyline cu fill cyan-fade dedesubt.

Window borderless cu chrome custom

Aplicatia nu foloseste chrome-ul Windows-ului (titlebar gri standard). In loc, are controale proprii (minimize/maximize/close) integrate in design. Pentru drag, exista zone invizibile pe top — apuci de oriunde, fereastra se misca. Double-click maximizeaza. Pe Windows 11 are background AcrylicBlur.

Particle field animat

Control custom care deseneaza ~60 particule cyan plutind in sus, cu fade-in/fade-out natural si blur subtle pe Skia. Implementat ca un singur Render() la 60 FPS pe canvas — mult mai eficient decat 60 controale Avalonia animate XAML.

CAPITOLUL 9

Ghid pentru demo

Pasii recomandati pentru demo in fata profesorului. Ai 5-10 minute, fiecare moment conteaza.

■ Pregatire (inainte de demo)

Pornit Windows-ul cu SQL Server activ (verifica in Services.msc: MSSQLSERVER trebuie sa fie *Running*). Daca e prima oara, ruleaza `setup-db.ps1` ca DB-ul sa fie populat cu date demo. Inchide aplicatii care fac zgomot vizual sau audio.

■ Deschide proiectul in Rider

Aratata profesorului structura: solution explorer cu folderele `database/`, `src/ITCareHelpdesk.App/` si in cadrul lui `Controls / Models / Services / ViewModels / Views`. Mentioneaza ca arhitectura este pe niveluri (MVVM + Services + Repositories).

■ Apasa F5 — porneste aplicatia

Asteapta sa apara splash-ul cu logo si particule. Atrage atentia ca splash-ul nu este cosmetic — el verifica conexiunea la SQL Server, si daca DB-ul nu raspunde, ar afisa eroarea exacta cu connection string-ul folosit.

■ Login Window — login normal

Demonstreaza autentificarea cu `admin / admin123`. Mentioneaza ca parola este hash-uita cu SHA256 si comparata in baza de date prin `sp_Login`. Daca gresesti parola de 5 ori, te blocheaza 15 minute.

■ Login Window — sign-up cu OTP

Da log out, apoi click pe "Creeaza un cont nou". Completeaza formularul, primesti un cod OTP intr-un toast in coltul dreapta. Lipeste-l (Ctrl+V pe prima casuta — se distribuie automat pe toate sase). Cont creat. Demonstreaza astfel **OTP**, **anti-bruteforce**, **sign-up cu validari**.

■ Dashboard

Dupa login, prima pagina. Comenteaza ce vezi: 6 KPI-uri (carduri colorate), donut chart cu distributia statusurilor, sparkline cu trendul de 7 zile, top 5 tehnicieni, tichete critice, distributia pe categorii cu bare orizontale. Click pe "■ Reincarca" — totul se reincarca paralel din DB.

■ Tichete — creare

Sidebar → Tichete. Lista cu tichete active, filtre. Click pe + **Tichet nou**. Dialog modal — completeaza titlu, descriere, alege client, categorie, prioritate, tip. Apasa "Creeaza tichet". Toast verde cu numarul nou. Lista se

reincarca automat. Mentioneaza ca sp_DeschideTichet genereaza
automat numarul (TKT-YYYY-NNNNN) si ataseaza SLA.

■ Rapoarte — export PDF

Sidebar → Rapoarte. Trei carduri (PDF/Word/Excel). Click pe PDF → file picker → salveaza. Deschide fisierul rezultat — arata-i layout-ul cu header brand ITCare, KPI strip, tabel zebra cu pill-uri colorate (rosu pentru CRITICAL, portocaliu HIGH, etc.), footer cu paginare.

■ Clienti, Tehnicienii, Asset-uri, Istoric

Demonstreaza-le pe rand. Clientii ca galerie de carduri. Tehnicienii ca trading cards cu metrici. Asset-urile ca tabel cu filtru pe tip echipament. Istoricul ca timeline cu linia verticala si puncte cyan.

■ Cod sursa — quick tour

Deschide in Rider: `SplashViewModel.cs` — comenteaza `Task.WhenAll` pentru incarcare paralela; `DatabaseService.cs` — wrapper SQL minimal; `01_main.sql` — schema, view-uri, proceduri stocate; `App.axaml.cs` — Dependency Injection container.

■ Inchidere demo

Marcheaza ca proiectul este pe GitHub (privat momentan pentru contextul de companie). Mentioneaza directii viitoare: integrare cu EVE-NG pentru topologii de retea, AI assistant pentru triaj automat, heatmap analytics.

Sfat final pentru demo

Nu te grabi. Lasa ochiul profesorului sa absoarba fiecare ecran cateva secunde. Daca te intreaba ceva, raspunde scurt si concret. Daca nu stii ceva, spune "nu sunt sigur, dar pot verifica" — e mai onest si mai impresionant decat sa improvizezi.

CAPITOLUL 10

Intrebari probabile + raspunsuri

Ce ar putea sa te intrebe profesorul si cum sa raspunzi clar, fara sa pari ca recitezi.

Î. De ce ai ales C# si nu Python/Java?

R. C# imi permite sa fac aplicatii desktop performante cu interfata grafica nativa, fara browserul intermediar pe care l-ar cere o aplicatie web. Pe langa asta, ecosistemul .NET este matur — am acces la pachete robuste pentru SQL Server, generare PDF, MVVM, fara sa-mi fac eu de la zero infrastructura.

Î. De ce Avalonia si nu WPF?

R. WPF este vechi (lansat in 2006), exclusiv Windows si in mod oficial Microsoft nu mai investeste activ in el. Avalonia este modern, cross-platform (acelasi cod ar putea rula pe Linux sau macOS), are performanta superioara pe controale custom (datorita motorului Skia) si dezvoltare activa. In plus, sintaxa AXAML este aproape identica cu XAML, deci experienta WPF nu se pierde.

Î. De ce nu ai folosit Entity Framework (un ORM)?

R. Trei motive: (1) Practica scolara cerea sa demonstrez ADO.NET si proceduri stocate explicit, ca sa intelegem ce se intampla la nivel de SQL. (2) EF Core adauga overhead-ul mapper-ului si genereaza query-uri uneori sub-optimale. (3) Pentru o aplicatie desktop monolitica de marime medie, ADO.NET direct e mai simplu de depanat — la review-uri sau debug, vezi exact ce SQL se executa.

Î. De ce hash SHA256 simplu pentru parole si nu bcrypt/argon2?

R. Recunosc ca SHA256 simplu (fara salt) **nu este productie-grade**. Pentru practica scolara este suficient sa demonstrez conceptul. In productie reala, as folosi bcrypt sau argon2 cu salt unic per user — care sunt rezistente la atacuri cu hardware specializat (rainbow tables, GPU brute-force). Aplicatia ar trebui doar sa schimbe functia HashPassword si sa migreze hash-urile existente la noua schema.

Î. Ce este MVVM si de ce l-ai ales?

R. Model-View-ViewModel este un pattern arhitectural care separa interfata (View) de logica de prezentare (ViewModel) si de date (Model). Avantaje: codul se citește si se testeaza usor, pot schimba UI-ul fara sa ating logica, pot adauga o aplicatie mobila re folosind ViewModel-urile. L-am ales pentru ca este standardul de facto pentru aplicatii Avalonia/WPF.

Î. Cum functioneaza Dependency Injection in proiect?

R. In App.axaml.cs, am un container `IServiceCollection` in care inregistrez toate serviciile (singleton-uri precum `DatabaseService`) si ViewModels (transient — instanta noua la fiecare navigare). Cand cineva are nevoie de o instanta, o cere din container care i-o livreaza cu toate dependentele rezolvate automat. Asta inseamna ca nu trebuie sa scriu manual `"new DatabaseService(new SqlClient(...))"` peste tot.

Î. Cum se calculeaza daca un tichet a depasit SLA?

R. View-ul SQL `vw_TicheteActive` calculeaza la fiecare `SELECT`: `CASE WHEN DATEDIFF(HOUR, data_deschidere, GETDATE()) > sla.timp_rezolvare_ore THEN 1 ELSE 0 END AS sla_depasi`. Nu salvam acest flag in tabela — il calculam in fly, deci nu se invecbeste. Aplicatia citește view-ul si afiseaza coloana cu fundal rosu daca flag-ul este 1.

Î. Ce sunt particulele de pe splash/login? Sunt utile?

R. Sunt un control custom (`ParticleField.cs`) care deseneaza ~60 puncte cyan ce plutesc in sus. Estetic, da aplicatiei o atmosfera "mission control". Functional, particulele creeaza miscare in fundal care subtil ghideaza ochiul spre zona principala — un trick UX. Implementarea este pe Skia direct, deci 60 FPS fara impact pe CPU.

Î. De ce ai facut chart-urile (donut, sparkline) custom si nu cu o biblioteca?

R. Bibliotecile de charting (LiveCharts, ScottPlot) sunt 5-15 MB si vin cu un look default care nu se potriveste cu paleta noastra. Implementarea custom are ~80 de linii per chart, este controlata exact pe paleta brandului, si nu adauga dependinte. Pentru doua chart-uri simple, costul de mentenanta e mic.

Î. Cum ai testat aplicatia?

R. Testarea automata (unit tests) nu este obligatorie pentru practica, dar am structurat codul astfel incat sa fie testabil: ViewModels nu cunosc UI-ul direct, Services nu cunosc ViewModels, Repositories nu cunosc Services. Daca ar fi sa adaug teste, as folosi xUnit cu Moq pentru mock-uri pe Services. Manual am testat fiecare flow (login, signup, creare tichet, export PDF) la fiecare modificare semnificativa.

Î. Daca ai timp, ce ai adauga?

R. Trei lucruri prioritare: **(1)** Detail drawer pe tichet — momentan listezi si inchizi, dar nu poti "intra" intr-un tichet ca sa vezi descrierea completa, comentariile si istoricul. **(2)** Integrare cu EVE-NG (laborator de retele) pentru topologii vizuale legate de tichete — diferenta majora. **(3)** Integrare AI: la creare tichet, un buton "Sugereaza" care propune categorii + prioritate folosind Claude API pe baza titlului si descrierii.

Î. Cum poate altcineva sa ruleze proiectul?

R. Pasii sunt in fisierul `README.md`: (1) Instalati .NET 8 SDK + SQL Server Express. (2) Ruleaza scriptul `database/setup-db.ps1` care creeaza baza de date si o populeaza cu seed. (3) Deschide solution-ul in Rider/VS, F5. (4) Login cu `admin/admin123`. Tot proiectul este pe GitHub (privat momentan).

ANEXE

Glosar si comenzi utile

Glosar — termeni tehnici explicati

ADO.NET

Componenta .NET pentru acces la baze de date relationale (SQL Server, MySQL etc.). Foloseste obiecte `SqlConnection`, `SqlCommand`, `SqlDataReader`.

AXAML

Dialect XML al Avalonia pentru descrierea UI-ului. Echivalentul XAML din WPF.

Binding

Mecanism prin care o proprietate din ViewModel este "legata" de o proprietate UI — cand una se schimba, cealalta se actualizeaza automat.

DI	Dependency Injection. Tehnica prin care un obiect primeste dependintele lui din afara, nu le creeaza singur.
DTO / POCO	Data Transfer Object / Plain Old C# Object. Clase simple care contin doar date, fara comportament.
FK	Foreign Key. Coloana intr-o tabela care "pointeaza" catre cheia primara a altei tabele.
KPI	Key Performance Indicator. Metrice cheie urmarite (ex. tichete deschise, satisfactie medie).
MVVM	Model-View-ViewModel. Pattern de organizare a codului care separa UI de logica de prezentare.
ORM	Object-Relational Mapper. Biblioteca care mapeaza tabelele SQL la obiecte (Entity Framework, Dapper).
OTP	One-Time Password. Cod de unica folosinta trimis prin SMS/email pentru verificare suplimentara.
Repository	Pattern: clasa care abstractizeaza accesul la o entitate din DB.
SLA	Service Level Agreement. Acord pe timpii maximi de raspuns si rezolvare per tip contract si prioritate.
SP / Stored Procedure	Functie scrisa in SQL care traieste in baza de date si poate fi apelata de aplicatie.
Trigger	Functie SQL care se executa automat la INSERT/UPDATE/DELETE pe o tabela.
View (SQL)	Interogare pre-impachetata care poate fi folosita ca o tabela. Nu stocheaza date proprii.
XAML	eXtensible Application Markup Language. Format XML pentru descrierea UI in WPF, Avalonia, UWP.

Comenzi utile (cheatsheet)

Verificare SQL Server activ	<code>Get-Service -Name MSSQL*</code>
Rulare scripturi DB (PowerShell)	<code>cd database; .\setup-db.ps1</code>
Rulare scripturi DB manual	<code>sqlcmd -S localhost -E -i 01_main.sql</code>
Build proiect	<code>cd src\ITCareHelpdesk.App; dotnet build</code>
Rulare proiect	<code>dotnet run --project src\ITCareHelpdesk.App</code>
Reset DB complet	<code>sqlcmd -S localhost -E -Q "DROP DATABASE IF EXISTS ITCareHelpdesk"</code>
Verificare connection string (test rapid)	<code>sqlcmd -S localhost -E -Q "SELECT @@VERSION"</code>
Git — primul commit + push	<code>git init -b main; git add .; git commit -m "first"; git push -u origin main</code>
Vezi cate tichete sunt in DB	<code>sqlcmd -S localhost -d ITCareHelpdesk -E -Q "SELECT COUNT(*) FROM Tichete"</code>

Conturi demo (pentru autentificare)

admin / admin123 (rol Admin) **manager.ops / manager123** (rol Manager) **mihai.popescu / tech123** (rol Technician) **demo / demo123** (rol Technician — fara tehnician_id atasat)